ERRONKA 7 · ADMINISTRAZIO ATALA

IAren Erabileraren Txostena

Powerbank Alokairu Sistema · Taldearen prozesua, kodea eta hausnarketa

Txosten honek taldeak IA nola erabili duen azaltzen du Powerbank Alokairu Sistemaren web aplikazioa hobetzeko: erabilitako tresnak eta ereduak, prompt esanguratsuak, sortutako kodea, aurkitutako akatsak, IAren proposamenetan egindako zuzenketak eta IA erabili gabe edo erabilita lan egiteko moduen arteko hausnarketa.

Tresnak
OpenCode, DeepSeek V4.1 Flash eta Edge DevTools

Promptak
Sei eskaera nagusi eta haien emaitzak

Kodea
Bateria, FLIP, mapa-sinkronia eta auto-berritzea

Hausnarketa
IAren mugak, zuzenketak eta taldearen rola

Egileak	Xabier Lazcoz eta Aitor Mindeguia
Bertsioa	1.0
Data	2026ko azaroa
Hizkuntza	Euskara
Proiektua	Erronka 7 · Administrazio Atala

Powerbank Inc. · Donostia / Zarautz

Txosten hau taldearen lana da. IAren erantzunak ez dira itsuan aplikatu: kodea irakurri, probatu eta zuzendu da. Taldekide guztiek funtzionalitateak eta kodea azaltzeko gai izan behar dute.

Aurkibidea

Laburpena 4

Dokumentuaren egitura 4

1. Testuingurua eta erabilitako IA tresnak 5

1.1 Proiektuaren testuingurua 5

1.2 Erabilitako IA tresnak eta ereduak 5

1.3 Nola erabili zen IA 5

2. IAri egindako prompt esanguratsuak 7

2.1 Prompt nagusiak 7

2.2 Prompten irakurketa teknikoa 7

2.3 Prompt-ingeniaritzako ikasgaiak 7

3. IAren bidez sortutako edo aldatutako kodea 9

3.1 Aldaketen ikuspegi orokorra 9

3.2 movimiento.js: orrialde-arteko transizioak eta FLIP 9

3.3 Bateria dinamikoa: edukiera erreala 10

3.4 Mapa eta zerrendaren arteko sinkronia 10

3.5 Auto-berritzea eta aldaketen detekzioa 11

3.6 CSS: mugimendua eta estiloa 11

3.7 HTML aldaketak 12

4. Aurkitutako akatsak eta arazoak 13

4.1 Akatsen laburpena 13

4.2 Akatsik esanguratsuena: animation-fill-mode eta transform 13

4.3 Akats semantikoa: bost segmentu finko 14

5. IAren proposamenetan egindako zuzenketak 15

5.1 Proposamenak eta erabakiak 15

5.2 Taldeak egin behar izan zituen zuzenketa nagusiak 15

6. Hausnarketa: IA erabili gabe eta erabilita 16

6.1 Konparazioa 16

6.2 Zer irabazi genuen 16

6.3 Zer arrisku ikusi genituen 16

6.4 Nola lan egin behar du taldeak IArekin 16

6.5 Taldearen konpromisoa 17

7. Ondorioak 18

Eranskin A: Defentsa-galderen gida 19

Eranskin B: Egiaztapenak eta glosarioa 20

B.1 Egindako probak 20

B.2 Bateriaren egiaztapen-taula 20

B.3 Glosarioa 20

B.4 PDFa berriro sortzea 21

Laburpena

Txosten honek **Powerbank Alokairu Sistema** proiektuan taldeak IA (Adimen Artifiziala) nola erabili duen dokumentatzen du. Bereziki, erabiltzailearen web aplikazioaren kargaguneen orria (`Kargaguneak.html`) hobetzeko egindako lana hartzen da oinarri: mugimendu- eta animazio-geruza berria, estazioen benetako edukiera erakusten duen bateria-barra, mapa eta zerrendaren arteko sinkronizazioa eta datuen auto-berritzea.

Txostenak sei atal nagusi ditu: erabilitako tresnak eta ereduak, prompt esanguratsuak, sortutako edo aldatutako kodea, aurkitutako akatsak, IAren proposamenetan egindako zuzenketak eta IA erabili gabe edo erabilita lan egiteko moduen arteko hausnarketa. Horrez gain, taldekideek kodea azaldu ahal izateko galdera gida bat eta egiaztapen-eranskinak gehitu dira.

> **Ohar garrantzitsua.** Txosten hau ez da IAren erantzunen bilduma bat. IAk proposatutako kodea irakurri, probatu, zuzendu eta ulertu egin da. Taldeko kide guztiek sortutako funtzionalitateak eta kodea azaltzeko gai izan behar dute; horretarako, txostenaren amaieran defentsa-galderen gida bat dago.

Dokumentuaren egitura

Atala	Edukia
1	Testuingurua eta erabilitako IA tresnak eta ereduak
2	IAri egindako prompt esanguratsuak eta haien emaitzak
3	IAren bidez sortutako edo aldatutako kodearen azalpena
4	Aurkitutako akatsak, sintomak eta konponbideak
5	IAren proposamenetan egindako zuzenketak
6	IA erabilita eta erabili gabe lan egiteko moduen hausnarketa
A	Defentsa-galderen gida eta egiaztapenak
B	Aldatutako fitxategien laburpena eta glosarioa

1. Testuingurua eta erabilitako IA tresnak

1.1 Proiektuaren testuingurua

Sistema hiru osagai nagusik osatzen dute: `web/` karpetako HTML/CSS/JavaScript aplikazioak (erabiltzailearen interfazea), `src/` karpetako Angular administrazio-panelak eta `api_erronka/` karpetako FastAPI zerbitzuak, PostgreSQL datu-basearen gainean. Lan honetan erabiltzailearen web aplikazioaren kargaguneen orria hobetu da, baina betiere APIaren datu errealak erabiliz eta lehendik zeuden funtzionalitateak hautsiz gabe.

Helburua ez zen soilik "animazio gehiago" jartzea. Helburua zen mugimenduak **sistema ulertzen laguntzea**: estazio bakoitzak zenbat powerbank dituen erakustea, datuak aldatzen direnean nabaritzea, eta mapan zein zerrendan estazio bera seinalatzea. Horregatik, IAren proposamenak beti negozio-arauen eta datu errealen aurka egiaztatu dira.

1.2 Erabilitako IA tresnak eta ereduak

Tresna	Eredua / bertsioa	Erabilera	Mugak
OpenCode (agentea, CLI)	DeepSeek V4.1 Flash	Kodea irakurri, proposatu eta editatzeko; komandoak exekutatzeko; probak eta egiaztapenak gidatzeko	Ez du proiektuaren testuingurua bere kabuz ezagutzen; erabiltzaileak eman behar dio
Microsoft Edge (headless) + Chrome DevTools Protocol	Edge 2026 bertsioa	DOM, estilo kalkulatuak eta pantaila-argazkiak egiaztatzeko; JavaScript erroreak atzemateko	Nabigatzailearen mende dago; ez du erabiltzailearen ingurune erreala guztiz ordezkatzen
Node.js	26.8.2	`node --check` bidez JavaScript fitxategien sintaxia balidatzeko	Sintaxia baino ez du egiaztatzen, ez portaera
FastAPI + uv + PostgreSQL	Proiektuko bertsioak	APIa abiarazi eta datu errealak lortzeko	Datu-basea martxan egon behar da
VS Code Live Server	—	`web/` karpeta zerbitzatzeko (5500 portua)	Cache-arazoak sor ditzake, beraz `?v=` erabili da
Git	—	Aldaketen kontrola eta `git diff` bidez berrikuspena	Ez du kodea automatikoki balidatzen

IA ez zen erabili irudiak sortzeko, ez eta datu-basearen diseinua automatikoki erabakitzeko ere. IAren papera **garapen-laguntzaile** batena izan zen: irakurri, proposatu, idatzi, probatu eta azaldu. Azken erabakiak, probak eta zuzenketak taldearenak izan ziren.

1.3 Nola erabili zen IA

Lana modu iteratiboan egin zen:

1. Erabiltzaileak behar bat azaldu zuen, askotan hizkera arruntean.
2. Agenteak fitxategiak irakurri eta egitura ulertu zuen.
3. Proposamen bat egin zen, kodea idatzi aurretik.
4. Kodea editatu eta sintaxia egiaztatu zen.
5. Nabigatzailean probatu zen, datu errealekin.
6. Erabiltzaileak emaitza aztertu eta zuzenketa eskatu zuen.

7. Akatsak zuzendu eta berriro probatu zen.

Puntu garrantzitsua: **IA ez zen inoiz "itsuan" aplikatu**. Prompt bakoitzaren ondoren kodea irakurri zen, eta zalantzak zeuden tokietan proba espezifikoak egin ziren.

2. IAri egindako prompt esanguratsuak

2.1 Prompt nagusiak

Hurrengo taulan lan-saio honetako benetako eskaerak bildu dira. Jatorrizko testua mantendu da, eta ondoan zer lortu zen eta zer ikasi zen azaldu da.

#	Jatorrizko prompta (laburdua)	Asmoa	Emaitza
1	"inicia la api para que pueda ver la pagina"	Ingurunea abiaraztea	APIa 8000 portuan eta web-a 5500ean abiarazi ziren; datu-basearen egoera egiaztatu zen
2	"que cosas le añadirias a la pagina? alguna animacion con movimiento? que se te ocurre?"	Ideiak eta arkitektura-proposamena	IAren analisi bat: zer animazio mota duen zentzua, zer den decoratiboa eta zer funtzionala
3	"hazlo"	Proposamenaren zati bat inplementatzea	1. eta 2. faseak: transizioak, FLIP, mapa-sinkronia, bateria eta auto-berritzea
4	"tambien en el index pone 4 de 6 puntos o 3 de 4 puntos que no ponga puntos tiene que poner powerbanks"	Testuaren zuzenketa semantikoa	Etiketak "de 6 powerbanks" formatura aldatu ziren, singular/pluralarekin
5	"si solo hay 1 powerbank que solo se vea 1 nivel o que la barra muestre realmente todas las powerbanks posibles en el punto si maximo tiene 2 pues que salgan 2 barras grandes"	Bateriaren segmentu kopurua edukierara egokitzea	Bateria dinamikoa: segmentu kopurua estazioaren edukiera da
6	"en index se muestran las powerbanks que hay con una barra de bateria [...] no que todas tengan 5 [...] que cambie el tamaño de las barras para que ocupen todas lo mismo pero que muestre cuantas hay y puede haber realmente"	Bateriaren diseinuaren doikuntza eta cache-arazoa	Segmentuak edukieraren arabera, guztiek zabalera bera, eta `?v=2` cachearen aurka

2.2 Prompten irakurketa teknikoa

Prompt laburrak eraginkorrak izan ziren, baina **testuinguru gehiago** eman zitzaionean emaitza hobea izan zen. Adibidez:

- "hazlo" bezalako prompt batek ez du zehazten zein fitxategi aldatu, baina aurretik egindako proposamena onartzea dakar. Horrelako promptak arriskutsuak dira testuingurua galdu bada.
- "no que todas tengan 5" bezalako esaldiak **negozio-arau bat** adierazten zuen: estazioen edukiera aldakorra da. Hori da IAk bere kabuz asmatu behar ez zuena; datuetatik irakurri behar zuen.
- "que cambie el tamaño de las barras para que ocupen todas lo mismo" bezalako esaldiak **diseinu-baldintza** bat ematen zuen: barra guztiek zabalera bera izan behar dute, baina barruko segmentuek edukiera islatu behar dute.

2.3 Prompt-ingeniaritzako ikasgaiak

1. **Eman testuingurua**: fitxategi-izenak, unean ikusten den portaera eta espero den emaitza.
2. **Eman baldintza negoziozkoak**: adibidez, "estazioak 2, 3, 4, 5 edo 6 powerbank izan ditzake".
3. **Eskatu egiaztapena**: "probatu nabigatzailean" edo "egiaztatu ez duela errore ematen".
4. **Egin zuzenketa txikiak**: prompt luze batek baino emaitza hobea ematen du iterazio laburrek.

5. **Ez kopiatu itsuan**: IAk kodea sortzen duenean, galdetu zergatik aukeratu duen soluzio hori.

3. IAren bidez sortutako edo aldatutako kodea

3.1 Aldaketen ikuspegi orokorra

Fitxategia	Mota	Gutxi gorabeherako aldaketa	Eginkizuna
`web/js/movimiento.js`	Berria	~150 lerro	Orrialde-arteko transizioak, FLIP utilitateak eta mugimendu murriztuaren detekzioa
`web/js/index.js`	Aldatua	~300 lerro	Bateria dinamikoa, FLIP, mapa-sinkronia, auto-berritzea eta zuzenketak
`web/css/lista.css`	Aldatua	~167 lerro	Bateriaren estiloa, halo/tilt efektua, markatzaileen nabarmendzea eta `cardIn` zuzenketa
`web/css/base.css`	Aldatua	~37 lerro	Orrialde-sarrera/irteera eta `prefers-reduced-motion`
`web/Kargaguneak.html`	Aldatua	3 lerro	`movimiento.js` gehitu, `aria-live` eta cache-bustinga
`web/index.html`	Berria	~80 lerro	Hasiera orria: informazio orokorra, estatistikak eta erreserbarako CTA
`web/Erreserbak.html`, `web/Erabiltzailea.html`, `web/Login.html`, `web/puesto-Carga.html`	Aldatuak	1 lerro bakoitzean	`movimiento.js` gehitu

3.2 movimiento.js: orrialde-arteko transizioak eta FLIP

Fitxategi berri honek bi gauza nagusi egiten ditu. Lehenik, barneko esteketan klik egitean gorputza leuntzen du eta ondoren nabigatzen du; bigarrenik, FLIP (First-Last-Invert-Play) teknikaren utilitateak eskaintzen ditu zerrendak berriz margotzen direnean elementuak leun mugitzeko.

```
function capturarPosiciones(contenedor, selector) {
  const posiciones = new Map();
  contenedor.querySelectorAll(selector).forEach((elemento) => {
    posiciones.set(elemento.dataset.clave, elemento.getBoundingClientRect());
  });
  return posiciones;
}
```

Funtzio honek elementu bakoitzaren posizioa gordetzen du `data-clave` atributuaren bidez. Gero, zerrenda berriz margotu ondoren, `reproducirFLIP` funtzioak desberdintasuna kalkulatzen du eta `element.animate()` bidez konpontzen du:

```
const dx = antes.left - despues.left;
const dy = antes.top - despues.top;

elemento.animate(
  [
    { transform: `translate(${dx}px, ${dy}px)` },
    { transform: "translate(0, 0)" },
  ],
  { duration: 360, easing: "cubic-bezier(0.22, 1, 0.36, 1)" },
);
```

Horrela, erabiltzaileak bilaketa edo ordena aldatzen duenean, txartelak ez dira "saltoka" aldatzen; posizio berrira irristatzen dira. Mugimendua `prefers-reduced-motion` errespetatuz desaktibatzen da.

3.3 Bateria dinamikoa: edukiera erreala

Hasierako IAren proposamenean bost segmentu finko zeuden. Taldeak akatsa atzeman zuen: estazio guztiek ez dute bost powerbank. Azken soluzioak estazioaren `karga_postuak` kopurua erabiltzen du segmentu kopurua zehazteko:

```
const totalSegmentos = Math.max(1, estado.total);
const encendidos = Math.min(estado.libres, totalSegmentos);
bateria.style.setProperty("--segmentos", String(totalSegmentos));

for (let indice = 0; indice < totalSegmentos; indice += 1) {
  const segmento = crearElemento("span", "station-battery-segment");
  if (indice < encendidos) segmento.classList.add("is-on");
  bateria.appendChild(segmento);
}
```

CSSak aldagai hori erabiltzen du zutabe kopurua zehazteko:

```
.station-battery {
  display: grid;
  grid-template-columns: repeat(var(--segmentos, 5), minmax(0, 1fr)) 4px;
}
```

Ondorioa: 2 powerbank dituen estazioak bi segmentu handi erakusten ditu; 6 dituenak sei segmentu. Guztiek zabalera total bera daukate, baina zelulen tamaina edukierara egokitzen da. Gainera, karga-maila hutsa den segmentuak gris edo anbar kolorez margotzen dira, edukiera antzeman dadin.

3.4 Mapa eta zerrendaren arteko sinkronia

Bi egitura bereizi erabiltzen dira: `marcadoresPorEstacion` eta `tarjetasPorEstacion`. Bietan estazioaren `id`-a gakoa da. Txartelaren gainean sagua jartzean, markatzailea aktibatzen da; markatzailearen gainean jartzean, berriz, txartela nabarmentzen da.

```
function resaltarEstacion(poste) {
  const marcador = marcadoresPorEstacion.get(String(poste.id));
  if (!marcador) return;

  marcador.getElement()?.classList.add("is-active");

  if (!mapa || !mapaVisible) return;
  mapa.flyTo([Number(poste.latitud), Number(poste.longitud)], 14, { duration: 0.7 });
}
```

`mapaVisible` aldagaia `IntersectionObserver` bidez eguneratzen da. Horrela, mapa pantailan ikusten ez denean ez da alferrikako `flyTo` animaziorik egiten.

3.5 Auto-berritzea eta aldaketen detekzioa

Index orriak 30 segundoro datuak berritzen ditu, baina ez du erabiltzailearen lana eteten. Hiru babes daude:

1. **Ikusgarritasunaren kontrola**: fitxa ezkutatuta badago, ez da eskaerarik egiten.
2. **Aldaketen detekzioa**: estazio bakoitzaren sinadura konparatzen da (`egoera` , `libres` eta karga-postuen kopurua). Aldaketarik ez badago, ez da zerrenda berriz margotzen.
3. **Filtroen kontserbazioa**: udalerri hautatua eta bilaketa-testua mantentzen dira.

```
function claveEstado(poste) {
  return `${poste.egoera}|${poste.libres}|${(poste.karga_postuak || []).length}|${poste.udalerria_id}`;
}

temporizadorRefresco = window.setInterval(() => {
  if (document.visibilityState === "visible") Datuak({ silencioso: true });
}, INTERVALO_REFRESCO);
```

Aldaketa bat dagoenean, txartelak `.is-updated` klasea jasotzen du eta distira arin bat erakusten du. Horrela, erabiltzaileak badaki datuak eguneratu direla, baina ez du pantaila osoa berriz kargatzen.

3.6 CSS: mugimendua eta estiloa

`lista.css` fitxategian hainbat bloke gehitu ziren:

- **Bateria**: segmentuak, karga-distira eta edukieraren koloreak.
- **Halo efektua**: `::before` elementu bat, `--mx` eta `--my` aldagaiekin saguen posizioa jarraitzen duena.
- **Tilt efektua**: txartelaren transformazio arina (`rotateX` / `rotateY`), 3 gradu ingurukoa.
- **Markatzaileen nabarmentzea**: `.is-active` eta `.is-linked` klaseak.
- `cardIn` **zuzenketa**: `animation-fill-mode: both` → `backwards` .

```
.estacion-item:hover,
.estacion-item:focus-visible,
.estacion-item.is-linked {
  transform: perspective(1000px) rotateX(var(--rx)) rotateY(var(--ry)) translateY(-2px);
}
```

3.7 HTML aldaketak

Bost HTML fitxategietan `movimiento.js` gehitu zen, `<head>` barruan. `Kargaguneak.html` fitxategian, gainera, `aria-live="polite"` gehitu zitzaion emaitza-kontagailuari, eta `lista.css` zein `index.js` fitxategien URLetan `?v=2` gehitu zen cache zaharrak saihesteko.

4. Aurkitutako akatsak eta arazoak

4.1 Akatsen laburpena

#	Sintoma	Kausa	Nola atzeman zen	Konponbidea
1	Hover egin arren txartela ez zen okertzen, haloak bai	`cardIn` animazioak `both` fill-mode zuen eta `transform: none` betirako aplikatzen zuen	`getComputedStyle()` bidez `matrix(1,0,0,1,0,0)` ikusi zen, nahiz eta `:hover` aktibo egon	`animation-fill-mode: backwards` erabili
2	Bateria guztiek 5 segmentu zituzten	IAk kopuru finkoa suposatu zuen	Taldeak eskakizuna zehaztu zuen: edukiera aldakorra da	Segmentu kopurua `estado.total`-etik kalkulatu
3	"4 de 6 puntos" testua	Hasierako etiketak "puntos" erabiltzen zuen	Erabiltzaileak berariaz zuzendu zuen	"de 6 powerbanks" eta singular/plural egokia
4	Udalerri-filtroa berritzean galtzen zen	`renderizarMunicipios()`-ek hautatutako balioa ez zuen berrezartzen	Auto-berritzea ezarri zenean pentsatu zen	Hautatutako balioa gorde eta berrezarri
5	APIak huts eginez gero zerrenda hutsik geratzen zen	Refresco automatikoak errore-bidea hartzen zuen	Kodearen berrikuspenean atzeman zen	Modu isilak datuak mantentzen ditu; errorea ez da bistaratzen
6	Mapa 30 segundoro zoom-era itzultzen zen	`fitBounds` bakoitzeko exekutatzen zen	Auto-berritzearen proban ikusi zen	Puntuen sinadura konparatu eta soilik aldaketarekin egokitu
7	Nabigatzaileak bertsio zaharra erakusten zuen	JS/CSS cachean gordeta	Erabiltzailearen feedbackak erakutsi zuen	`?v=2` cache-bustinga eta Ctrl+F5 gomendioa
8	Lehen kargako animazioak eta aldaketen distira gainjartzen ziren	`.animate-in` eta `.is-updated` aldi berean	Kodearen berrikuspenean	Lehen karguan aldaketen zerrenda hutsik utzi

4.2 Akatsik esanguratsuena: `animation-fill-mode` eta `transform`

Akats hau da txosteneko adibiderik argiena IAren kodea **ez** dela itsuan onartu behar erakusteko. Hover efektua ez zen funtzionatzen: haloak bai, baina txartelaren tilt-ak ez. Lehen begi-kolpean CSS arazo bat zirudien, baina kodea ondo zegoen.

Diagnostikoa nabigatzailean egin zen:

```
getComputedStyle(tarjeta).transform
// "matrix(1, 0, 0, 1, 0, 0)"
```

Emaitza hori identitate-matrizea da: ez zegoen ez perspektibarik, ez biraketarik. Kausa `.estacion-item.animate-in` araua zen:

```
/* Okerra */
animation: cardIn 0.45s cubic-bezier(0.22, 1, 0.36, 1) both;
```

`both` erabiltzean, animazioaren azken fotograma (`transform: none`) amaitu ondoren ere aplikatzen da. CSS animazioek propietate arruntek baino lehentasun handiagoa dute, beraz `:hover`-eko transformazioa ez zen inoiz aplikatzen. Konponbidea:

```
/* Zuzena */
animation: cardIn 0.45s cubic-bezier(0.22, 1, 0.36, 1) backwards;
```

Konpondu ondoren, `getComputedStyle()`-k `matrix3d(...)` itzuli zuen, `translateY(-2px)` eta biraketa txikiak barne. Hau da IAren kodea probatzearen garrantziaren adibide garbia: kodea sintaktikoki zuzena zen, baina portaeran okerra.

4.3 Akats semantikoa: bost segmentu finko

IAren lehen soluzioak "bateria" baten itxura eman nahi zuen bost segmenturekin. Hori ohiko diseinu bat da, baina ez zetorren bat datuekin: estazioek 2, 3, 4, 5 edo 6 karga-postu izan ditzakete. Akatsa ez zen sintaxi-akatsa, **eredu mental okerra** baizik.

Taldeak datuak egiaztatu zituen APIan:

Estazioa	Edukiera	Libres
Kargagunea Zubieta	6	4
Kargagunea Donostia Centrico	2	0
Kargagunea Easo	4	3
Kargagunea Urbieta	3	0
Kargagunea Gros	3	0
Kargagunea Zarautz Antiguo	2	1
Kargagunea Zarautz Amara	2	0
Kargagunea Zarautz Egia	4	0
Kargagunea Zarautz Aiete	5	3
Kargagunea Zarautz Intxaurrondo	3	2

Datu horiek erabiliz, segmentu kopurua edukierara egokitu zen. Emaitza nabigatzailean egiaztatu zen: 6 edukierako estazioak 6 zelula, 2 edukierakoak 2 zelula, eta abar.

5. IAren proposamenetan egindako zuzenketak

5.1 Proposamenak eta erabakiak

IAren proposamena	Erabakia	Arrazoia
View Transitions API erabiltzea orrialde-arteko animaziorako	Ez zen erabili; JS/CSS fade-a erabili zen	Chrome/Edge-ren mende zegoen, eta Leaflet maparekin bateragarritasun-arriskuak zituen
Txartelaren klik-ak mapara `flyTo` egitea	Ez zen horrela egin; hover/focus erabili zen	Txartelaren klik-ak estazioaren orrira nabigatu behar du; bi ekintzak ezin dira nahastu
Klik bakoitzean markatzailea irekitzea	Ez; soilik nabarmentzea eta maparen ikusgaitasunaren araberako `flyTo`	Erabiltzailearen nabigazioa ez etetea
Txartelaren barruan "mapa" botoia jartzea	Ez zen egin	`<a>` baten barruan `<button>` jartzea HTML baliogabea da eta irisgarritasun-arazoa sortzen du
Bost segmentuko bateria finkoa	Edukieraren araberako segmentuak	Datuak eta negozio-arauak errespetatzea
Karga-ikono gehigarria (rayo)	Distira efektua soilik	Elementu bisual gehiegi ez pilatzea
Datu guztiak 30 segundoro berrirudikatzea	Aldaketak badaude soilik	Fokua, filtroak eta errendimendua zaintzea
Animazioa bilaketaren tekla bakoitzean	Debounce (140 ms) eta FLIP	Idazten ari den bitartean errendimendua ez gutxitzea

5.2 Taldeak egin behar izan zituen zuzenketa nagusiak

1. **Edukiera**: 5 segmentu finkotik `estado.total`-era pasatzea.
2. **Testua**: "puntos" hitza "powerbanks" hitzarekin ordeztea, singular/pluralarekin.
3. **Fill-mode**: `both` → `backwards`, hover-a ez blokeatzeko.
4. **Filtroen kontserbazioa**: udalerri hautatua ez galtzea.
5. **Errore isila**: APIak huts egiten badu, datuak ez ezabatzea.
6. **Maparen zoom-a**: `fitBounds` soilik puntuak aldatzen direnean.
7. **Cachea**: `?v=2` gehitzea eta erabiltzaileari Ctrl+F5 gomendatzea.
8. **Animazioen talka**: lehen karguan `.is-updated` ez aplikatzea.

Zuzenketa hauek ez ziren automatikoki sortu. IAk kodea proposatu zuen, baina taldeak probatu, neurtu eta erabaki zuen zein den soluzio egokia. Horregatik da garrantzitsua kodea ulertzea: IAk ez daki, adibidez, estazio batek 2 powerbank edo 6 dituenik, ez bazaio esaten edo datuetatik irakurtzen ez badu.

6. Hausnarketa: IA erabili gabe eta erabilita

6.1 Konparazioa

Alderdia	IA erabili gabe	IA erabilita
Abiadura	Geldoagoa: dokumentazioa bilatu, proba/errore asko	Azkarragoa: kodea eta probak lehenago sortzen dira
Esplorazioa	Soluzio ezagunetara mugatuagoa	Aukera gehiago: FLIP, WAAPI, `IntersectionObserver`, etab.
Kalitatea	Giza ezagutzaren mende	IAren kodea zuzena izan daiteke sintaktikoki, baina okerra semantikoki
Ikaskuntza	Prozesu geldoagoa, baina sakona	Azkarrago ikasten da, baina arriskua dago "kopia" egiteko
Akatsak	Ohikoak, baina ulertzeko errazagoak	Akats sotilak (adibidez, `animation-fill-mode`)
Dokumentazioa	Eskuz idatzi behar da	IAk zirriborroa azkar egin dezake, baina taldeak berrikusi behar du
Erantzukizuna	Taldearena	Taldearena, are gehiago: kodea ulertu behar da

6.2 Zer irabazi genuen

- **Denbora**: animazio-geruza oso bat (transizioak, FLIP, bateria, sinkronia, auto-berritzea) askoz denbora laburragoan prestatu zen.
- **Ideia berriak**: IAk FLIP teknika, `IntersectionObserver` bidezko ikusgaitasun-kontrola eta Web Animations APIa proposatu zituen, eskuz agian ez genituzkeenak erabiliko.
- **Proba automatizatuak**: CDP bidezko proba txikiak idatzi ziren, eta horiek akats bat (transformazioa blokeatuta) argi erakutsi zuten.
- **Dokumentazio hobea**: kodea azaltzeko egitura eta glosarioa sortu ziren.

6.3 Zer arrisku ikusi genituen

- **Egiaztapenik gabeko kodea**: IAk kodea ondo idatzita eman dezake, baina datuekin bat ez datorrena. Bost segmentuen akatsa horren adibidea da.
- **Uste okerrak**: IAk ez daki proiektuaren negozio-arauak berak; erabiltzaileak eman behar dizkio.
- **Iritzuzko konponbideak**: zenbait proposamen (View Transitions, klik bidezko `flyTo`) ez ziren egokiak proiektu honetarako; taldeak baztertu egin behar izan zituen.
- **Mantentze-arriskua**: IAk sortutako kodea ulertzen ez bada, zaila da etorkizunean aldatzea. Horregatik, kodea azaltzeko gida gehitu da.

6.4 Nola lan egin behar du taldeak IArekin

1. **Prompt argia**: zer aldatu, non eta zergatik.
2. **Kodearen irakurketa**: IAk emandako kodea lerroz lerro ulertu.
3. **Proba erreala**: nabigatzailean, datu errealekin, ez soilik sintaxia egiaztatuz.
4. **Datuen egiaztapena**: APIaren erantzunak eta datu-basearen balioak konparatu.
5. **Irisgarritasuna eta mugimendu murriztua**: beti kontuan hartu.

6. **Azalpena**: taldekide batek ezin badu kodea azaldu, kodea ez da onartzen.

6.5 Taldearen konpromisoa

Taldekide guztiek gai izan behar dute, gutxienez, hurrengoak azaltzeko:

- Bateriaren segmentuak nola kalkulatzen diren eta zergatik den `estado.total` erabiltzen den.
- FLIP teknikaren urratsak: lehen posizioa, bigarren posizioa, aldea eta animazioa.
- Zergatik ez den `animation-fill-mode: both` erabiltzen `cardIn` animazioan.
- Nola sinkronizatzen diren mapa eta zerrenda.
- Nola funtzionatzen duen auto-berritzeak eta zergatik ez dien erabiltzailearen filtroei eragiten.
- Nola errespetatzen den `prefers-reduced-motion`.

7. Ondorioak

IA tresna lagungarria izan da, baina **ez da taldearen ordezkoa**. Lan honetan IAk kodea azkarrago idazten lagundu zuen, baina akats garrantzitsuenak taldeak aurkitu zituen: edukiera finkoaren akats semantikoa eta animazioaren `fill-mode` akats sotila. Bi akats horiek erakusten dute IAren emaitza beti egiaztatu behar dela, bai datuekin, bai nabigatzailean.

Emaitza: kargaguneen orriak bateria-barra dinamikoa du, estazio bakoitzaren edukiera erreala erakusten duena; txartelak leun mugitzen dira filtratzean; mapak eta zerrendak elkarrekin erantzuten dute; eta datuak 30 segundoro berritzen dira erabiltzaileari eragotzi gabe. Kodea CSS/JavaScript hutsean oinarritzen da, kanpoko mendekotasun berririk gabe, eta mugimendu murriztua errespetatzen du.

Etorkizunerako hobekuntza posibleak: erreserba berriaren baieztapen animatua, QRaren eskaneatze-lerroa eta erreserbaren kontagailu bisuala. Horiek datozen faseetan landu daitezke, betiere proba eta azalpenarekin.

Eranskin A: Defentsa-galderen gida

Taldekideek defentsan galdetu dezaketen galderak eta erantzunaren gakoak:

1. **Zergatik ez da bost segmentu finko erabiltzen?** Estazioen edukiera aldakorra delako (2 eta 6 artean). Segmentu kopurua `estado.total` da, eta CSSak `--segmentos` aldagaia erabiltzen du.
2. **Zer da FLIP?** First-Last-Invert-Play: aldaketaren aurreko eta ondorengo posizioa neurtu, aldea kalkulatu eta `transform: translate()` bidez animatu.
3. **Zergatik `backwards` eta ez `both`?** `both` erabiliz gero, animazioaren azken fotogramak `transform: none` betirako aplikatzen du eta hover-a blokeatzen du. `backwards`-ek ez du animazioa amaitu ondoren propietatea bahitzen.
4. **Nola egiaztatu zen akats hori?** `getComputedStyle(tarjeta).transform` identitate-matrizea zela ikusi zen, nahiz eta `:hover` eta `is-tilting` aktibo egon. Konpondu ondoren `matrix3d(...)` itzuli zuen.
5. **Nola funtzionatzen du auto-berritzeak?** 30 segundoro, fitxa ikusgai badago, `Datuak({ silencioso: true })` deitzen da. Datuak aldatu ez badira, ez da zerrenda berriz margotzen.
6. **Zergatik ez da mapa berriz marrazten aldaketarik gabe?** Markatzaileen sinadura konparatzen da. Aldaketarik ez badago, markagailuak eta zoom-a mantentzen dira.
7. **Nola sinkronizatzen dira mapa eta zerrenda?** `marcadoresPorEstacion` eta `tarjetasPorEstacion` mapak erabiltzen dira, `id`-a gakoa izanik. Hover/focus-ek klaseak gehitzen ditu; `flyTo` soilik mapa ikusgai dagoenean.
8. **Zer gertatzen da APIak huts egiten badu auto-berritzean?** Datuak mantentzen dira; ez da zerrenda husten. Erabiltzaileak ez du informazioa galtzen.
9. **Nola errespetatzen da mugimendu murriztua?** CSSko `@media (prefers-reduced-motion: reduce)` eta JavaScripteko `prefiereMenosMovimiento()` bidez.
10. **Zergatik gehitu zen `?v=2`?** Nabigatzaileak JS/CSS zaharrak cachean gordetzeko aukera saihesteko. Horrela, aldaketak berehala ikusten dira.
11. **Zer aldatu zen testuan?** "puntos" hitza kendu eta "powerbanks" jarri zen, singular/plural egokiarekin.
12. **Zer da `aria-live="polite"`?** Pantaila-irakurleei emaitza-kontagailuaren aldaketak iragartzeko, erabiltzaileari eten gabe.

Eranskin B: Egiaztapenak eta glosarioa

B.1 Egindako probak

Proba	Tresna	Emaitza
JavaScript sintaxia	`node --check`	OK
Index orriaren karga	Edge headless (CDP)	10 estazio, 10 markatzaile
Txartel → mapa sinkronia	CDP `pointerenter`	Markatzailea aktibatzen da
Mapa → txartel sinkronia	CDP `mouseover`	Txartela nabarmentzen da
Filtroa	CDP bilaketa	Emaitza zuzena, errore gabe
Refresco isila	`Datuak({ silencioso: true })`	Datuak mantentzen dira
Orrialde-arteko transizioa	CDP klik	`is-leaving` eta nabigazioa
Beste orrialdeak	CDP	Login, Erreserbak, Erabiltzailea eta puesto-Carga errore gabe
Bateriaren edukiera	CDP neurketa	6→6, 5→5, 4→4, 3→3, 2→2

B.2 Bateriaren egiaztapen-taula

Estazioa	Edukiera	Libres	Segmentuak	Piztuta
Kargagunea Zubieta	6	4	6	4
Kargagunea Easo	4	3	4	3
Kargagunea Zarautz Aiete	5	3	5	3
Kargagunea Zarautz Intxaurrondo	3	2	3	2
Kargagunea Zarautz Antiguo	2	1	2	1
Kargagunea Donostia Centrico	2	0	2	0
Kargagunea Urbieta	3	0	3	0
Kargagunea Gros	3	0	3	0
Kargagunea Zarautz Amara	2	0	2	0
Kargagunea Zarautz Egia	4	0	4	0

B.3 Glosarioa

- **API**: aplikazioen arteko komunikazio-interfazea. Hemen FastAPI bidezkoa.
- **CDP**: Chrome DevTools Protocol, nabigatzailea programatik kontrolatzeko.
- **DOM**: dokumentuaren objektu-eredua, HTMLaren zuhaitza.
- **FLIP**: First-Last-Invert-Play, posizio-aldaketak leun animatzeko teknika.
- **WAAPI**: Web Animations API, JavaScript bidezko animazioak.
- `prefers-reduced-motion`: erabiltzaileak animazio gutxiago nahi dituen adierazteko CSS media query-a.

- `aria-live`: pantaila-irakurleek edukia aldatzen denean iragartzeko atributua.
- **Cache-busting**: fitxategi baten URLari bertsioa gehitzea, cache zaharrak saihesteko.
- `fill-mode`: animazioak amaitu ondoren zein balio mantentzen duen zehazten duen CSS propietatea.

B.4 PDFa berriro sortzea

Proiektuaren erro-karpetatik:

```
uv run --with markdown --with playwright --with pypdf python docs/build_ia_pdf.py
```

Scriptak lehendik instalatuta dagoen Chrome/Edge erabiltzen du PDFa sortzeko, eta `docs/assets/estiloa.css` estiloa aplikatzen du.

> **Taldearen oharra.** Txosten honen zirriborroa IAren laguntzaz prestatu da, baina taldeak edukia berrikusi, egiaztatu eta osatu behar du. Defentsan, taldekide bakoitzak atal baten azalpena eman beharko luke, eta kodea zuzenean azaltzeko prest egon.